MLDL
PRACTICAL 9

Name: Payal Dara
Class: D15C
Roll no: 56
Batch: C

AIM:-

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

---

1. Dataset Source

Dataset Name: Airline Passengers Dataset
Source: Public Time Series Dataset (commonly used for LSTM)
Link:
https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv

---

2. Dataset Description

This dataset contains monthly totals of international airline passengers.

Dataset Characteristics

- Time Period: 1949–1960
- Records: 144
- Feature: Number of passengers
- Type: Time series data

Target

- Predict future passenger values

---

3. Mathematical Formulation of RNN/LSTM

RNN processes sequential data using hidden states.

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

LSTM improves RNN using gates:

- **Forget Gate**

$$f_t = \sigma(W_f [h_{t-1}, x_t])$$

- **Input Gate**

$$i_t = \sigma(W_i [h_{t-1}, x_t])$$

- **Output Gate**

$$o_t = \sigma(W_o [h_{t-1}, x_t])$$

---

4. Algorithm Limitations (RNN/LSTM)

- Computationally expensive
- Requires large data
- Training is slow
- Complex architecture
- Sensitive to hyperparameters

---

5. Methodology / Workflow

Dataset → Preprocessing → Sequence Creation
→ LSTM Model → Training → Prediction → Graph

---

6. Performance Analysis

Metrics

- Loss (MSE)
- Prediction accuracy (trend matching)

Interpretation

- Lower loss → better prediction
- Graph shows prediction vs actual values

---

7. Hyperparameter Tuning (LSTM)

Parameters Tuned

- Number of LSTM units
- Epochs
- Batch size

Effect

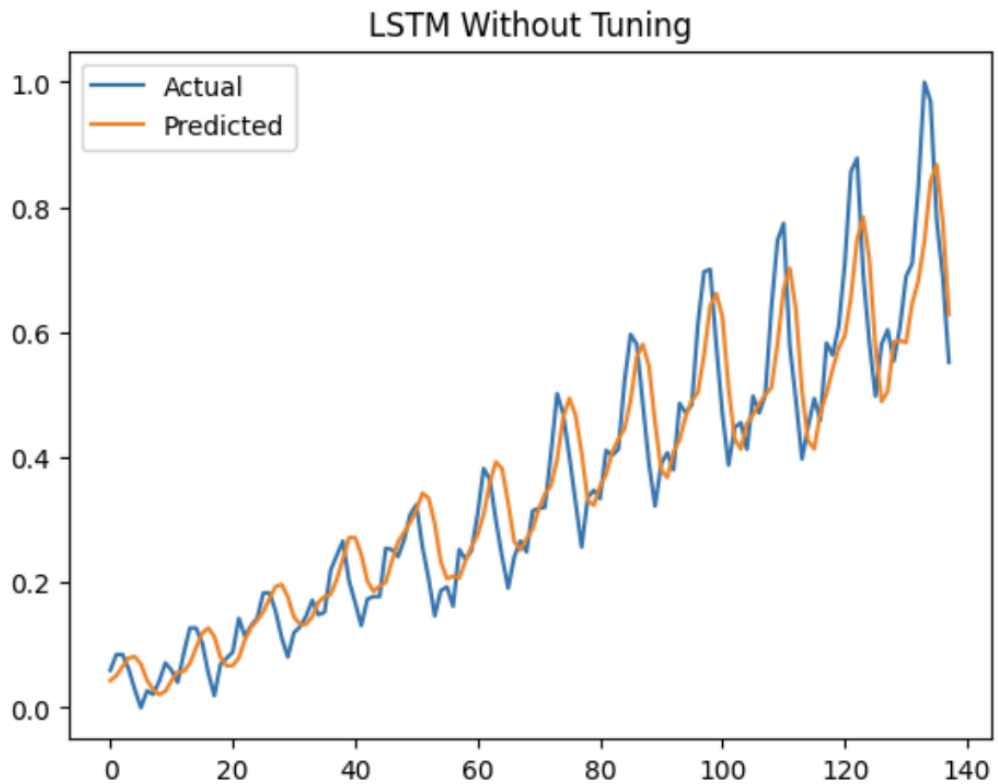
- Improves learning
- Better sequence modeling
- Reduces error

Code:-

Without tuning

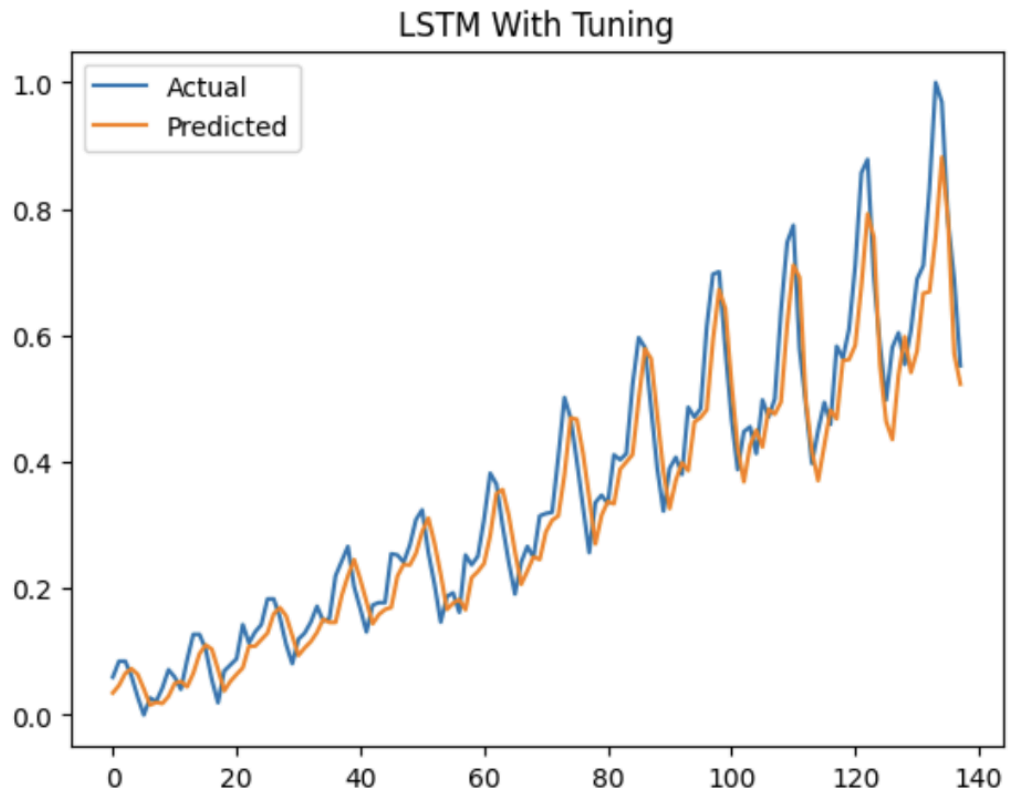
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Load dataset
url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.c
sv"
df = pd.read_csv(url)

data = df['Passengers'].values.reshape(-1,1)

# Scaling
scaler = MinMaxScaler()
data = scaler.fit_transform(data)

# Create sequences
def create_data(dataset, time_step=5):
    X, y = [], []
    for i in range(len(dataset)-time_step-1):
        X.append(dataset[i:(i+time_step), 0])
        y.append(dataset[i+time_step, 0])
    return np.array(X), np.array(y)

X, y = create_data(data, 5)

# Reshape
X = X.reshape(X.shape[0], X.shape[1], 1)

# Model
model = Sequential()
```

```
model.add(LSTM(50, activation='relu', input_shape=(5,1)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')

# Train
history1 = model.fit(X, y, epochs=50, batch_size=16)

# Prediction
pred1 = model.predict(X)

# Graph
plt.figure()
plt.plot(y, label='Actual')
plt.plot(pred1, label='Predicted')
plt.title("LSTM Without Tuning")
plt.legend()
plt.show()
```



With tuning:-

```
# Tuned Model

model2 = Sequential()
model2.add(LSTM(100, activation='relu', input_shape=(5,1)))
model2.add(Dense(1))

model2.compile(optimizer='adam', loss='mse')

# Train with different parameters
history2 = model2.fit(X, y, epochs=80, batch_size=8)

# Prediction
pred2 = model2.predict(X)

# Graph
plt.figure()
plt.plot(y, label='Actual')
plt.plot(pred2, label='Predicted')
plt.title("LSTM With Tuning")
plt.legend()
plt.show()
```



Interpretation:-
The graph compares actual and predicted passenger values. The tuned model follows the actual trend more closely, indicating better learning. Smaller differences between curves indicate better performance.

CONCLUSION:-

The graph compares actual and predicted passenger values. The tuned model follows the actual trend more closely, indicating better learning. Smaller differences between curves indicate better performance.